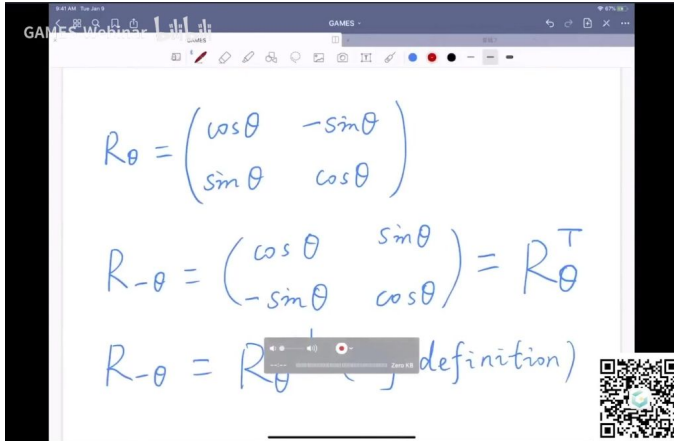


一、变换 00:07

1. 上节回顾 01:16

1) 二维情况下的变换 01:24

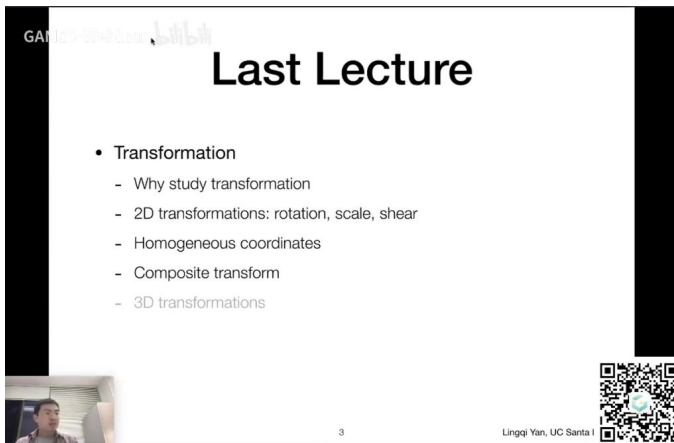


- 旋转矩阵形式：二维旋转 θ 角度可表示为 $R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$ ，其中 θ 为旋转角度。
- 负角度旋转：旋转 $-\theta$ 角度时，矩阵变为 $R_{-\theta} = \begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix}$ ，利用余弦偶函数和正弦奇函数性质推导得出。

2) 旋转矩阵的逆与转置的关系 02:54

- 关键性质：旋转矩阵的逆等于其转置，即 $R_{-\theta} = R_\theta^T = R_\theta^{-1}$ 。这是旋转矩阵特有的正交性质。
- 正交矩阵定义：满足 $A^T = A^{-1}$ 的矩阵称为正交矩阵，旋转矩阵是典型的正交矩阵实例。

3) 变换的应用与二维变换类型 03:46

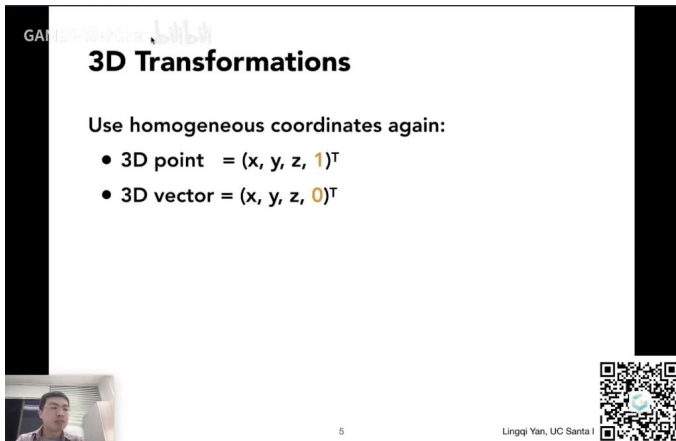


- 基本变换类型：包括旋转、缩放、切变三种基本线性变换。
 - 平移特殊性：平移变换无法表示为标准矩阵乘法形式，需要额外添加平移向量。
- #### 4) 齐次坐标的引入 04:21
- 解决方案：通过增加维度（二维变三维）实现统一表示，点表示为 $(x,y,1)$ ，向量表示为 $(x,y,0)$ 。
 - 矩阵扩展：变换矩阵扩展为 3×3 形式，最后一列为平移分量，实现线性变换和平移的统一处理。
- #### 5) 变换的合成与分解 05:09

- **组合原理**：通过矩阵乘法组合多个变换，遵循结合律但不满足交换律。
- **变换顺序**：先应用线性变换部分（左上 2×2 子矩阵），再应用平移分量（最后一列）。

2. 三维变换 05:24

1) 三维齐次坐标 06:37



3D Transformations

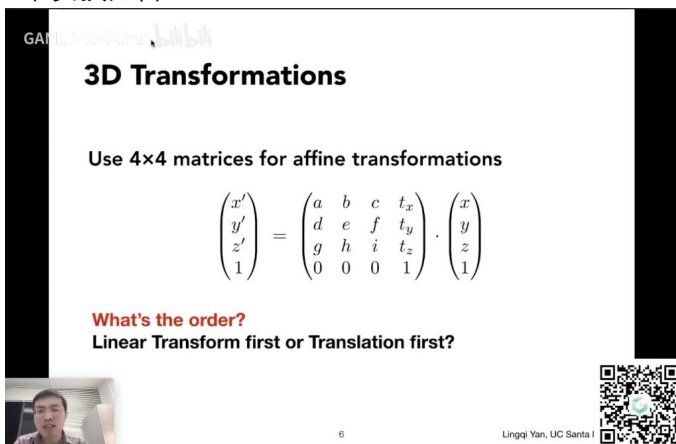
Use homogeneous coordinates again:

- 3D point = $(x, y, z, 1)^T$
- 3D vector = $(x, y, z, 0)^T$

5 Lingqi Yan, UC Santa I

- **坐标表示**：点表示为 $(x, y, z, 1)$ ，向量表示为 $(x, y, z, 0)$ ， $w \neq 0$ 时需进行归一化处理。
- **归一化规则**：齐次坐标 (x, y, z, w) 对应三维点为 $(x/w, y/w, z/w)$ 。

2) 四维变换矩阵 07:31



3D Transformations

Use 4×4 matrices for affine transformations

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & c & t_x \\ d & e & f & t_y \\ g & h & i & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

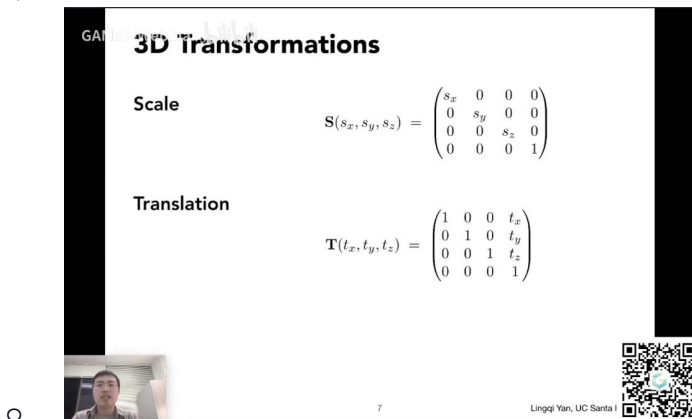
What's the order?
Linear Transform first or Translation first?

6 Lingqi Yan, UC Santa I

- **矩阵结构**： 4×4 矩阵中，左上 3×3 为线性变换，最后一列前三个元素为平移量，最后一行固定为 $(0, 0, 0, 1)$ 。
- **变换顺序**：与二维情况一致，先进行线性变换再进行平移。

3) 基本三维变换 08:34

- **缩放 08:38**



3D Transformations

Scale

$$S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Translation

$$T(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

7 Lingqi Yan, UC Santa I

- 矩阵形式: $S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \\ 0 & 0 & 0 \end{pmatrix}$, 允许各轴向独立缩放。

- 平移 08:56

- 矩阵特点: 左上 3×3 为单位矩阵, 平移量出现在最后一列前三个元素位置。

- 旋转 09:08

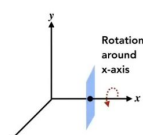
- 绕轴旋转矩阵 09:29

3D Transformations

Rotation around x-, y-, or z-axis

$$R_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_y(\alpha) = \begin{pmatrix} \cos \alpha & 0 & \sin \alpha & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \alpha & 0 & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_z(\alpha) = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 & 0 \\ \sin \alpha & \cos \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$


Rotation around x-axis

8 Lingqi Yan, UC Santa I

- X轴旋转: $R_x(\alpha)$ 保持x坐标不变, y-z平面进行二维旋转。
- Y轴旋转: $R_y(\alpha)$ 矩阵形式特殊, 因坐标系循环对称性导致sin项符号变化。
- Z轴旋转: $R_z(\alpha)$ 与二维旋转类似, 在x-y平面进行旋转。

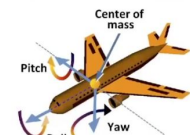
- 欧拉角 12:51

3D Rotations

Compose any 3D rotation from R_x, R_y, R_z ?

$$R_{xyz}(\alpha, \beta, \gamma) = R_x(\alpha) R_y(\beta) R_z(\gamma)$$

- So-called *Euler angles*
- Often used in flight simulators: roll, pitch, yaw



Center of mass

Pitch

Roll

Yaw

9 Lingqi Yan, UC Santa I

- 分解原理: 任意三维旋转可分解为绕x、y、z轴旋转的组合 $R_{xyz}(\alpha, \beta, \gamma)$ 。
- 飞行模拟类比: 对应roll (横滚)、pitch (俯仰)、yaw (偏航) 三种基本旋转。

- 罗德里格斯旋转公式 15:34

Rodrigues' Rotation Formula

Rotation by angle α around axis $\mathbf{n}$

$$R(\mathbf{n}, \alpha) = \cos(\alpha) \mathbf{I} + (1 - \cos(\alpha)) \mathbf{nn}^T + \sin(\alpha) \underbrace{\begin{pmatrix} 0 & -n_z & n_y \\ n_z & 0 & -n_x \\ -n_y & n_x & 0 \end{pmatrix}}_{\mathbf{N}}$$

How to prove this magic formula?

Check out the supplementary material on the course website!

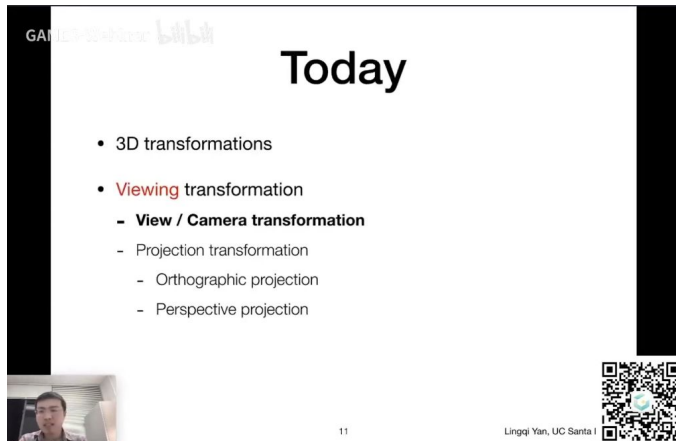
10 Lingqi Yan, UC Santa I

- 一般旋转表示: 绕任意过原点的轴 $\mathbf{n}$ 旋转 α 角度, 公式包含三项: 单位矩阵、外积项和反对称矩阵项。

- **任意轴旋转**：非过原点轴需先平移至原点，旋转后再平移回原位置。
- **四元数补充**：虽然课程不深入讲解，但提到四元数在旋转插值方面的优势。

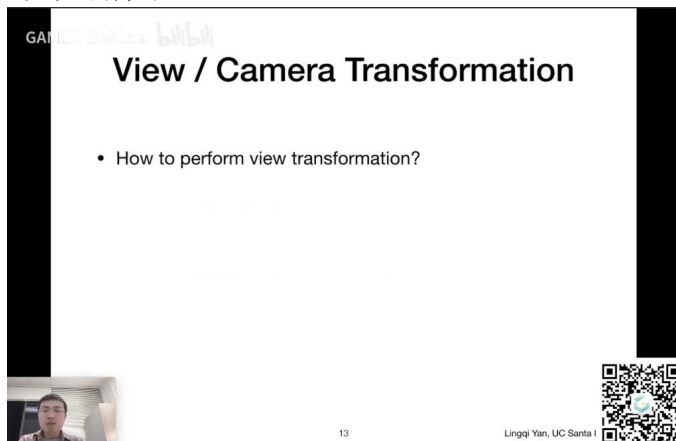
3. 视图变换 21:16

1) 视图变换定义 22:49



- **核心目的**：将三维空间中的物体转换为二维照片，包含视图变换和投影变换两个主要步骤
- **变换流程**：模型变换→视图变换→投影变换（简称MVP变换，Model View Projection）
- **相对运动原理**：相机与物体同步移动时，拍摄结果保持不变，这是视图变换的理论基础

2) 视图变换介绍 23:14



- **相机三要素**：
 - **位置**：相机中心点坐标 e
 - **观察方向**：gaze direction向量 g
 - **向上方向**：up vector向量 t （类比相机旋转角度）
- **标准约定**：将相机固定到原点 $(0,0,0)$ ，朝向负Z轴，向上为Y轴方向
- **数学本质**：通过矩阵变换将任意相机配置转换到标准位置

3) 视图变换操作 25:01

View / Camera Transformation

- M_{view} in math?
 - Let's write $M_{view} = R_{view}T_{view}$
 - Translate e to origin

$$T_{view} = \begin{bmatrix} 1 & 0 & 0 & -x_e \\ 0 & 1 & 0 & -y_e \\ 0 & 0 & 1 & -z_e \\ 0 & 0 & 0 & 1 \end{bmatrix}$$
 - Rotate g to -Z, t to Y, (g x t) To X
 - Consider its **inverse** rotation: X to (g x t), Y to t, Z to -g

WHY?

$$R_{view}^{-1} = \begin{bmatrix} x_g \times i & x_t & x_{-g} & 0 \\ y_g \times i & y_t & y_{-g} & 0 \\ z_g \times i & z_t & z_{-g} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \rightarrow R_{view} = \begin{bmatrix} x_g \times i & y_g \times i & z_g \times i & 0 \\ x_t & y_t & z_t & 0 \\ x_{-g} & y_{-g} & z_{-g} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Lingqi Yan, UC Santa I

变换步骤:

○ 平移: $T_{view} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$ 将相机中心移至原点

○ 旋转: 通过逆变换思路求解 R_{view} , 先求 $X \rightarrow g \times t, Y \rightarrow t, Z \rightarrow -g$ 的逆矩阵再转置

● 矩阵组合: $M_{view} = R_{view}T_{view}$ (注意矩阵乘法顺序)

● 关键技巧: 利用正交矩阵性质, 通过求逆变换矩阵的转置得到原旋转矩阵

4) 视图变换总结 35:06

View / Camera Transformation

- Summary
 - Transform objects together with the camera
 - Until camera's at the origin, up at Y, look at -Z

Lingqi Yan, UC Santa I

● 统一变换: 所有物体需与相机同步应用 M_{view} 变换

● 模型视图变换: 模型变换与视图变换常合并称为 ModelView 变换

● 后续流程: 完成视图变换后进入投影变换阶段

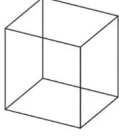
● 实际案例: 解释了"向上方向"的实际意义 (如相机旋转时顶部标记的方向变化)

4. 投影变换

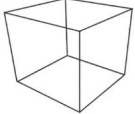
1) 投影类型对比

Projection Transformation

- Projection in Computer Graphics
 - 3D to 2D
 - Orthographic projection
 - Perspective projection



Orthographic projection



Perspective projection

Fig. 7.1 from *Fundamentals of Computer Graphics, 4th Edition*

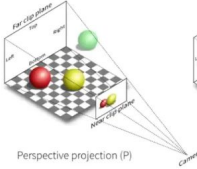
19 Lingqi Yan, UC Santa I

- 正交投影：
 - 特点：保持平行线平行，无近大远小效果
 - 应用：工程制图
 - 数学本质：相机距离无限远的极限情况
- 透视投影：
 - 特点：平行线会相交，产生近大远小效果
 - 应用：模拟人眼视觉（如一叶障目现象）
 - 典型表现：立方体的边延长线会相交于灭点

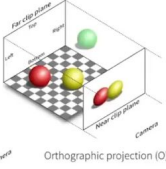
2) 投影数学原理

Projection Transformation

- Perspective projection vs. orthographic projection



Perspective projection (P)



Orthographic projection (O)

[s://stackoverflow.com/questions/36573283/from-perspective-picture-to-orthographic-picture](https://stackoverflow.com/questions/36573283/from-perspective-picture-to-orthographic-picture)

20 Lingqi Yan, UC Santa I

- 透视投影：
 - 视觉锥体：相机为顶点，近裁剪面和远裁剪面之间的四棱锥区域
 - 投影方式：将锥体内物体投影到近平面
- 正交投影：
 - 特殊情形：当相机无限远时，近/远平面大小相同
 - 投影结果：物体大小与距离无关（所有球体投影大小相同）

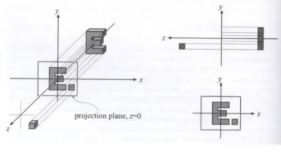
5. 投影变换 38:40

1) 正交投影 43:01

- 正交投影定义及理解

Orthographic Projection

- A simple way of understanding
 - Camera located at origin, looking at $-Z$, up at Y (looks familiar?)
 - Drop Z coordinate
 - Translate and scale the resulting rectangle to $[-1, 1]^2$

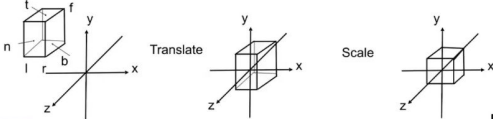


21 Lingqi Yan, UC Santa I

- 基本原理：将三维物体沿平行线投影到二维平面，忽略 z 坐标值
- 标准设置：相机位于原点 $(0,0,0)$ ，朝向负 Z 轴方向， Y 轴为向上方向
- 关键操作：
 - 直接丢弃所有点的 z 坐标
 - 将得到的 xy 平面图形平移并缩放到 $[-1,1]^2$ 范围
- 示例说明：三维空间中的字母E和小方块在不同 z 轴位置，丢弃 z 坐标后都会投影到同一 xy 平面位置
- 深度问题：这种简单方法无法区分物体前后关系（ z 轴信息完全丢失），需要后续处理
- 正交投影一般情况 45:27

Orthographic Projection

- In general
 - We want to map a cuboid $[l, r] \times [b, t] \times [f, n]$ to the "canonical (正则、规范、标准)" cube $[-1, 1]^3$



22 Lingqi Yan, UC Santa I

- 输入空间：任意长方体区域 $[l, r] \times [b, t] \times [f, n]$ （左/右、下/上、远/近）
- 输出空间：标准立方体 $[-1, 1]^3$ （Canonical Cube）
- 变换步骤：
 - 平移：将长方体中心移至原点
 - 缩放：将各轴长度缩放为2（对应 $[-1, 1]$ 范围）
- 坐标系特性：
 - 使用右手坐标系
 - 沿负 Z 轴观察导致 $n > f$ （近平面 z 值大于远平面）
 - 这是OpenGL等API采用左手系的原因之一
- 注意事项：
 - 变换会改变物体原始长宽比（后续需要视口变换修正）
 - 避免使用坐标系变换的解释方式（老师个人偏好）
- 正交投影矩阵 48:53

Orthographic Projection

- Transformation matrix?
 - Translate (**center** to origin) **first**, then scale (length/width/height to **2**)

$$M_{ortho} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 & 0 \\ 0 & 0 & \frac{2}{n-f} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -\frac{r+l}{2} \\ 0 & 1 & 0 & -\frac{t+b}{2} \\ 0 & 0 & 1 & -\frac{n+f}{2} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

24 Lingqi Yan, UC Santa

- 矩阵构成：由平移矩阵和缩放矩阵相乘得到

平移矩阵：

- $$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

缩放矩阵：

- $$\begin{bmatrix} \frac{2}{r-l} & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 \\ 0 & 0 & \frac{2}{n-f} \\ 0 & 0 & 0 \end{bmatrix}$$

○ 计算原理：

- 平移量取各轴中点坐标的负值
- 缩放因子=目标范围(2)/原始范围

○ 特殊处理：z轴缩放因子分母为 $n-f$ （因为 $n > f$ ）

Orthographic Projection

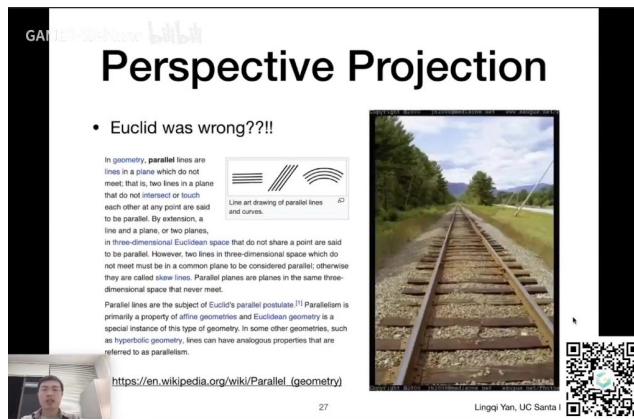
- Caveat
 - Looking at / along -Z is making near and far not intuitive ($n > f$)
 - FYI: that's why OpenGL (a Graphics API) uses left hand coords.

25 Lingqi Yan, UC Santa

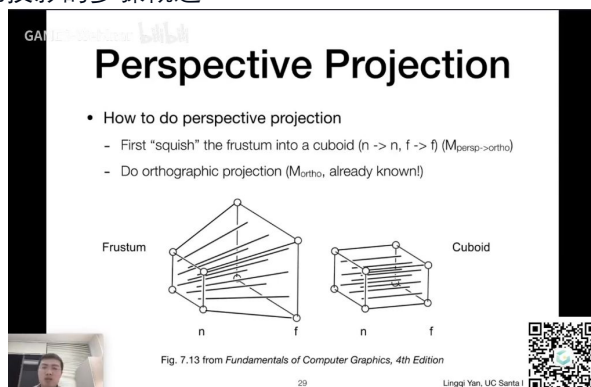
- 坐标系差异：
 - 负Z观察方向导致远近定义反直觉
 - 不同图形API可能采用不同坐标系（如OpenGL用左手系）
 - 作业中可能出现相反结果属于正常现象
- 变换顺序：必须先平移后缩放

2) 透视投影 50:46

- 欧式几何平行线定义 53:46



-
- **基本定义:** 在欧式几何中, 平行线指同一平面内永不相交的两条直线。扩展到三维空间, 平行平面指不共享任何点的两个平面。
- **特殊情况:**
 - **三维空间线关系:** 两条不相交的直线若不在同一平面内, 则称为"斜交线"(skew lines)
 - **非欧几何:** 在双曲几何等其他几何体系中, 存在类似平行线的概念但性质不同
- **透视投影反例:** 实际观察中平行铁轨会在远处"相交", 这是透视投影造成的视觉现象, 不违背欧式几何定义
 - **原因:** 透视投影将三维空间投影到二维平面时, 平行线在投影面上可能相交于"消失点"
 - **自然感知:** 人眼更适应透视投影效果, 正交投影的平行线反而显得不自然
- **正交投影与透视投影的区别 56:38**
 - **正交投影:**
 - **特征:** 保持平行关系, 远近物体大小相同
 - **应用:** 工程制图等需要精确测量的场景
 - **透视投影:**
 - **特征:** 模拟人眼视觉, 近大远小, 平行线可能相交
 - **数学基础:** 基于齐次坐标系统, (kx, ky, kz, k) 表示同一点 (x, y, z)
 - **关键公式:** 齐次坐标变换时, $(x, y, z, 1)$ 等价于 (xz, yz, z^2, z) ($z \neq 0$)
- **透视投影操作 58:33**
 - **透视投影的步骤概述**



-
- **两阶段处理:**
 - 将视锥体(frustum)挤压为长方体
 - 对长方体执行正交投影
- **矩阵表示:** $M_{persp} = M_{ortho} \times M_{persp \rightarrow ortho}$
- **投影过程拆解: 挤压与正交投影 59:22**

- **挤压操作目的:** 将远平面收缩至与近平面相同尺寸
- **正交投影作用:** 将标准化后的立方体投影到二维平面
- 挤压操作的具体规定 59:47

Perspective Projection

- How to figure out the third row of $M_{persp \rightarrow ortho}$
 - Any information that we can use?

$$M_{persp \rightarrow ortho} = \begin{pmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ ? & ? & ? & ? \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

- Observation: the third row is responsible for z'
 - Any point on the near plane will not change
 - Any point's z on the far plane will not change

不变性约束:

- 近平面所有点位置不变
- 远平面 z 坐标不变 (仅 xy 收缩)
- 远平面中心点位置完全不变

- 挤压操作中相似三角形关系 01:02:49

Perspective Projection

- In order to find a transformation
 - Recall the key idea: Find the relationship between transformed points (x', y', z') and the original points (x, y, z)

$$y' = \frac{n}{z}y$$

y 坐标变换: $y' = \frac{n}{z}y$

- **推导依据:** 通过侧视图中的相似三角形关系建立
- **物理意义:** 点越远(z 越大), y 坐标压缩越多

- 挤压操作中 y 坐标的变化 01:03:55

线性变化: 所有点的 y 坐标按 $\frac{n}{z}$ 比例压缩

边界验证:

- 当 $z = n$ 时 $y' = y$ (近平面不变)
- 当 $z = f$ 时 $y' = \frac{n}{f}y$ (远平面收缩)

- 透视投影中 x 坐标的变化 01:04:45

Perspective Projection

- In order to find a transformation
 - Find the relationship between transformed points (x', y', z') and the original points (x, y, z)

$$y' = \frac{n}{z}y \quad x' = \frac{n}{z}x \quad (\text{similar to } y')$$

- In homogeneous coordinates,

$$\begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \Rightarrow \begin{pmatrix} nx/z \\ ny/z \\ \text{unknown} \\ 1 \end{pmatrix} \xrightarrow{\text{mult. by } z} \begin{pmatrix} nx \\ ny \\ \text{still unknown} \\ z \end{pmatrix}$$

- 对称处理: $x' = \frac{n}{z}x$, 推导过程与y坐标完全对称
- 齐次坐标表示: $(x, y, z, 1) \rightarrow (nx, ny, ?, z)$

○ 透视投影矩阵的推导 01:06:33

Perspective Projection

- So the "squish" (persp to ortho) projection does this

$$M_{persp \rightarrow ortho}^{(4 \times 4)} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} nx \\ ny \\ \text{unknown} \\ z \end{pmatrix}$$

- Already good enough to figure out part of $M_{persp \rightarrow ortho}$

$$M_{persp \rightarrow ortho} = \begin{pmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ ? & ? & ? & ? \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad \text{WHY?}$$

■ 已知行确定:

- 第一行: $[n \ 0 \ 0 \ 0]$ (保证 $x' = nx$)
- 第二行: $[0 \ n \ 0 \ 0]$ (保证 $y' = ny$)
- 第四行: $[0 \ 0 \ 1 \ 0]$ (保证齐次坐标最后项为z)

○ 透视投影矩阵第三行的确定 01:08:38

- **z坐标特殊性:** 需要同时满足近远平面z值不变
- **矩阵形式:** 第三行必为 $[0 \ 0 \ A \ B]$ 形式

○ 透视投影矩阵第三行的计算 01:12:24

Perspective Projection

- What do we have now?

$$\begin{pmatrix} 0 & 0 & A & B \end{pmatrix} \begin{pmatrix} x \\ y \\ n \\ 1 \end{pmatrix} = n^2 \quad \rightarrow \quad An + B = n^2$$

- Any point's z on the far plane will not change

$$\begin{pmatrix} 0 \\ 0 \\ f \\ 1 \end{pmatrix} \Rightarrow \begin{pmatrix} 0 \\ 0 \\ f \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ f^2 \\ f \end{pmatrix} \quad \rightarrow \quad Af + B = f^2$$

■ 方程组建立:

- 近平面条件: $An + B = n^2$
- 远平面条件: $Af + B = f^2$

■ 解方程结果:

- $A = n + f$
- $B = -nf$

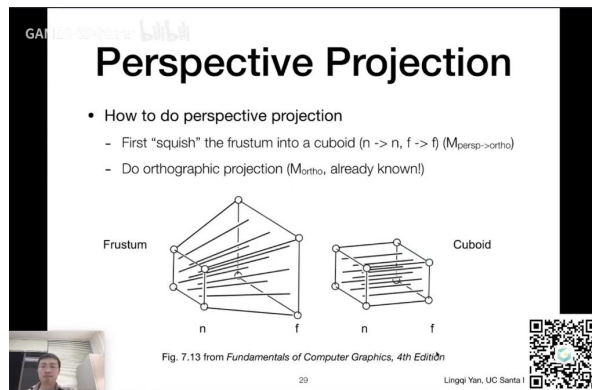
○ 透视投影矩阵的完成与后续操作 01:14:19

最终矩阵:

■
$$M_{persp \rightarrow ortho} = \begin{bmatrix} n & 0 & 0 \\ 0 & n & 0 \\ 0 & 0 & n + f \\ 0 & 0 & 1 \end{bmatrix}$$

■ 完整流程: 挤压后接正交投影矩阵 M_{ortho}

■ 例题1: 透视投影中z坐标的变化问题 01:15:12



问题分析:

- 近平面($z = n$)和远平面($z = f$)的 z 值不变
- 中间点 $z = \frac{n+f}{2}$ 的 z 值变化方向?

思考方向:

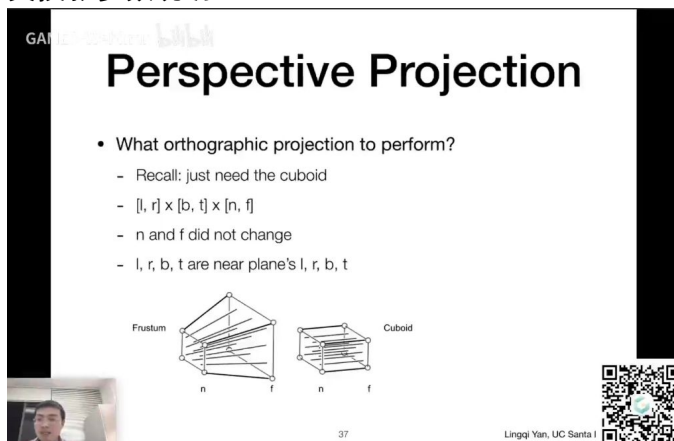
- 考察变换后 z 坐标表达式 $z' = (n+f) - \frac{nf}{z}$
- 分析中间点 z 值会偏向 n 还是 f

关键现象: 挤压操作会导致中间点 z 值向近平面移动

6. 透视投影矩阵求解

- **参数方程建立:** 通过 $An + B = n^2$ 和 $Af + B = f^2$ 建立方程组, 其中 n 和 f 分别表示近平面和远平面的 z 坐标值
- **求解过程:**
 - 由第一式得 $A = n + \frac{B}{n}$
 - 代入第二式解得 $B = -nf$
 - 最终得到 $A = n + f$
- **矩阵组合:** 透视投影矩阵 M_{persp} 由正交投影矩阵 M_{ortho} 和透视转正交矩阵 $M_{persp \rightarrow ortho}$ 相乘得到, 即 $M_{persp} = M_{ortho} M_{persp \rightarrow ortho}$

7. 正交投影参数说明



- **投影空间定义:** 使用规范立方体 $[l, r] \times [b, t] \times [n, f]$ 定义正交投影空间
- **参数特性:**
 - n 和 f 保持原始值不变
 - 左右下上参数(l, r, b, t)直接采用近平面边界值
 - 该立方体将透视视锥体转换为标准正交投影空间

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
二维变换矩阵	旋转、缩放、切变的矩阵表示方法	旋转矩阵的正交性 （逆=转置）	★★
齐次坐标	增加维度统一表示线性变换和平移	点($w=1$)与向量($w=0$)的区别	★★
三维变换	四维齐次坐标下的平移/旋转/缩放	绕y轴旋转矩阵符号特殊原因 （叉乘顺序）	★★★
视图变换	相机标准化（原点、-z朝向、y向上）	模型视图变换合并计算的相对运动原理	★★★★
正交投影	立方体映射到规范立方体 $[-1,1]^3$	远近平面定义 （ $n>f$ 因负z观察）	★★
透视投影	近大远小效果实现（Frustum→长方体）	中间点z值变化方向的思考题	★★★★★
投影矩阵推导	齐次坐标归一化（ n/z 因子）	近/远平面约束条件求矩阵参数	★★★★
罗德里格斯公式	任意轴旋转的矩阵表示	四元数在旋转插值的优势	★★★